

TP n°1

Résolution numérique d'un système linéaire par une méthode directe : application à un amortisseur

Méthodes Numériques pour la Mécanique – LST GM / Ingénieur GE – FST
Prof. M. JANANE ALLAH

8 septembre 2026

Objectifs pédagogiques

- Mettre en œuvre en MATLAB l'élimination de Gauss avec pivotage partiel, la descente/remontée, et la méthode de Gauss-Jordan (chapitre 2).
- Appliquer ces méthodes à un réseau d'amortisseurs linéaires et vérifier leur cohérence.
- Étudier expérimentalement le coût de calcul en fonction de la taille du système.
- Réutiliser une triangularisation déjà calculée pour plusieurs seconds membres (plusieurs cas de chargement).

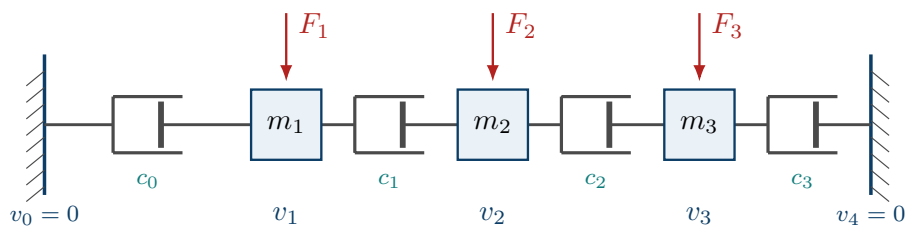
1 Modèle physique

On considère un banc d'essai composé de N masses alignées, reliées entre elles et à deux bâtis fixes par $N + 1$ amortisseurs linéaires de coefficients $c_0, c_1, \dots, c_N$ (loi $F = c \Delta v$). En régime permanent, l'équilibre des forces à chaque nœud i s'écrit

$$c_{i-1}(v_i - v_{i-1}) + c_i(v_i - v_{i+1}) = F_i,$$

avec $v_0 = v_{N+1} = 0$ (bâtis fixes). Pour $N = 3$, ceci donne le système étudié en cours :

$$C = \begin{pmatrix} c_0 + c_1 & -c_1 & 0 \\ -c_1 & c_1 + c_2 & -c_2 \\ 0 & -c_2 & c_2 + c_3 \end{pmatrix}, \quad Cv = F.$$



chaque masse est reliée à **exactement deux** amortisseurs $\Rightarrow$ matrice C **tridiagonale** et symétrique

Remarque

La matrice C est tridiagonale et symétrique (voir exercice d'entraînement du chapitre 2) : sa structure creuse en fait un bon exemple pour étudier le coût des différentes méthodes directes.

2 Manipulation 1 : élimination de Gauss sur le réseau à trois amortisseurs

On prend $c_0 = 2$, $c_1 = 3$, $c_2 = 1$, $c_3 = 4$ N.s/m et $F = (1, 4, 7)^T$ N.

MATLAB – à compléter

```

1 c0=2; c1=3; c2=1; c3=4;
2 C = [c0+c1 -c1 0; -c1 c1+c2 -c2; 0 -c2 c2+c3];
3 F = [1;4;7];
4
5 % 1) Ecrire une fonction gauss_pivot(A,b) implementant l'elimination
6 % de Gauss avec pivotage partiel (cf. algorithme du cours).
7 % 2) Resoudre Cv=F par cette fonction et par l'operateur natif C\F.
8 % 3) Comparer les deux solutions au residu ||C*v-F|| pres.
```

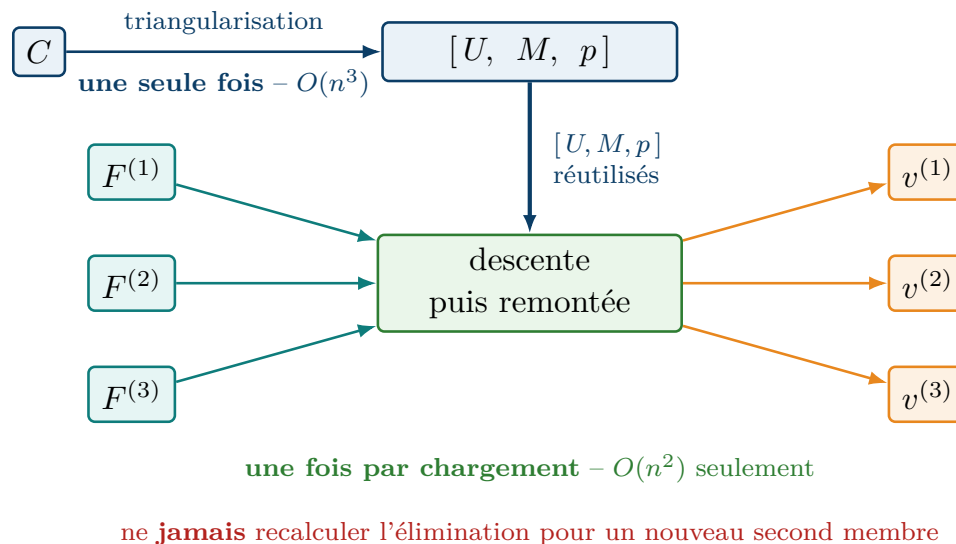
1. Compléter le script pour implémenter `gauss_pivot`, puis résoudre le système.
2. Vérifier que la solution obtenue coïncide (à 10^{-12} près) avec la solution manuelle du cours, $v = (2, 3, 2)^T$, et avec `C\F`.
3. Calculer le résidu $\|Cv - F\|_2$. Commenter l'ordre de grandeur obtenu.

3 Manipulation 2 : plusieurs cas de chargement

On souhaite maintenant étudier la réponse du même réseau à trois chargements différents :

$$F^{(1)} = (1, 4, 7)^T, \quad F^{(2)} = (0, 10, 0)^T, \quad F^{(3)} = (5, 5, 5)^T \quad (\text{N}).$$

1. Écrire une fonction `[U,M,p] = triangularise(C)` qui effectue la *seule* phase d'élimination de Gauss avec pivotage partiel (chapitre 2), sur C seule et sans second membre, et qui renvoie la matrice triangulaire supérieure U , les multiplicateurs m_{ik} et la permutation des lignes. Vérifier que les pivots obtenus sont bien 5, 2,2 et 4,5455, comme dans le calcul à la main du chapitre 2.
2. Écrire `y = descente_gauss(M,p,b)`, qui applique à un second membre les multiplicateurs déjà calculés. Pour chaque $F^{(k)}$, résoudre alors par `descente_gauss` puis `remontee`, **sans jamais recalculer l'élimination**.
3. Mesurer (`tic/toc`) le temps total de cette approche et le comparer au temps qu'aurait pris une résolution par `C\F(k)` à chaque fois (donc en refactorisant implicitement à chaque appel).



4 Manipulation 3 : étude de complexité

1. Pour $n \in \{100, 200, 400, 800, 1600, 3200\}$, générer une matrice aléatoire bien conditionnée $A = \text{rand}(n) + n \cdot \text{eye}(n)$ et un second membre aléatoire, puis mesurer le temps de résolution par $A \setminus b$ (prenez la *médiane* de quelques tirages, le temps d'une mesure isolée est bruité).
2. **Encadrez la mesure par `maxNumCompThreads(1)`**, puis restaurez la valeur initiale. Sans cette précaution, le multithreading des bibliothèques d'algèbre linéaire de MATLAB masque une partie de la croissance cubique et vous ne verrez jamais la loi attendue (question 5).
3. Tracer, en échelle log-log, le temps de calcul en fonction de n , superposé à une courbe de référence n^3 , et estimer la **pen**te par régression sur les grandes tailles seulement ($n \geq 400$).
4. Conclure sur l'accord entre la théorie ($O(n^3)$) et la mesure expérimentale.
5. Refaire la mesure *sans* forcer le monothread et comparer les deux pentes. Que mesure-t-on au juste dans chaque cas : la complexité de l'algorithme, ou la performance de la machine ?

5 Manipulation 4 : descente, remontée et Gauss-Jordan

On reprend le réseau à trois amortisseurs (Manipulation 1).

MATLAB – à compléter

```

1 % 1) Ecrire deux fonctions descente(L,b) et remontee(U,y) (cf.
   % chapitre 2).
2 % 2) A partir de [U,M,p]=triangularise(C) (Manipulation 2), résoudre
   % Cv=F
3 % par descente_gauss puis remontee.
4 % 3) Ecrire une fonction gauss_jordan(A,B) et l'utiliser pour :
5 % (a) résoudre Cv=F directement, (b) calculer C^-1 via
   % gauss_jordan(C, eye(3)).

```

1. Implémenter **descente** et **remontee**, et vérifier que la solution obtenue coïncide avec $v = (2, 3, 2)^T$ de la Manipulation 1.
2. Implémenter **gauss_jordan** et l'utiliser pour résoudre $Cv = F$ directement (sans étape de remontée séparée), puis pour calculer C^{-1} .

3. Vérifier numériquement que $CC^{-1} = I_3$ (à l'erreur d'arrondi près), et comparer C^{-1} obtenue à $\text{inv}(C)$.
4. Comparer, pour ce système 3×3 , le nombre de lignes de code nécessaires pour Gauss-Jordan par rapport à Gauss suivi d'une remontée. La simplicité d'implémentation justifie-t-elle, selon vous, le surcoût de calcul de Gauss-Jordan mentionné au chapitre 2 ?

6 Questions de synthèse

1. Pourquoi la matrice C d'un réseau d'amortisseurs est-elle toujours symétrique définie positive, quelles que soient les valeurs (positives) de $c_0, \dots, c_3$? (Indice : revoir la forme quadratique associée, chapitre 2.)
2. Dans quelle situation pratique (parmi celles rencontrées dans ce TP) la réutilisation d'une triangularisation déjà calculée apporte-t-elle le bénéfice le plus net ?
3. Si l'un des amortisseurs était défectueux et pouvait produire un coefficient c_i négatif, la matrice C resterait-elle en général inversible ? L'élimination de Gauss avec pivotage partiel (chapitre 2) pourrait-elle encore s'appliquer ?

Indices de correction

Manip. 1. La solution attendue est $v = (2, 3, 2)^T$: vérifiez-la avant d'aller plus loin. Contrôlez aussi le résidu $\|Cv - F\|_2$, qui doit tomber au niveau de $\varepsilon_{\text{mach}}$ ($\sim 10^{-15}$), pas à zéro exactement.

Manip. 2. `triangularise(C)` doit renvoyer les pivots 5, 2,2 et 4,5455, et la permutation $p = (1, 2, 3)$ – autrement dit **aucun échange n'est nécessaire ici**. C'est le même calcul que celui mené à la main au chapitre 2 : comparez.

Manip. 3. Faites varier n en échelle logarithmique et tracez le temps en `loglog` : c'est la *pente* qui doit valoir 3, pas les temps eux-mêmes. Deux effets faussent la mesure, et il faut neutraliser les deux : pour de petits n (≤ 200) le surcoût d'appel domine – au point que $n = 200$ peut paraître plus rapide que $n = 100$, ce n'est pas une erreur de votre part ; et sur tous les n , le **multithreading** rabat la pente vers 2,3 environ. En monothread et sur $n \geq 400$, attendez-vous à une pente de l'ordre de 2,6 à 2,8. Ne cherchez pas à retrouver exactement 3 : aucune machine réelle ne le donne.

Manip. 4. Les trois chemins (Gauss direct, descente/remontée, Gauss-Jordan) doivent donner le *même* v à 10^{-15} près. S'ils diffèrent au 3^e chiffre, c'est une erreur d'indice, pas d'arrondi.

Remarque

Le corrigé complet (réponses et script MATLAB) fait l'objet de documents séparés. **Cherchez d'abord** : un TP ouvert avec le corrigé à côté n'apprend rien.